AWS Failover Execution Plan

800 CarGuru — Active-Passive High Availability Setup

Start Date: April 2026 **Estimated Duration:** 3 Days **Executed By:** IT Department

---

Phase 1: Secure the Main Server (Day 1 — Morning)

> Goal: Ensure the current production server is properly protected before building the backup.

Step 1.1: Assign Elastic IP to Main Instance

- [ ] Go to EC2 → **Elastic IPs** → **Allocate Elastic IP address**
- [ ] Click **Allocate**
- [ ] Select the new IP → **Actions** → **Associate Elastic IP address**
- [ ] Choose main instance `SMF_NEW_CG` → **Associate**
- [ ] Note down:
  - Elastic IP: _____
  - Allocation ID: _____

Step 1.2: Update DNS to Elastic IP

- [ ] Login to **GoDaddy** → DNS for `800carguru.me`
- [ ] Update **A record** (`@`) to the new Elastic IP
- [ ] Update **A record** (`www`) to the new Elastic IP
- [ ] Set TTL to **600**
- [ ] Verify: `ping 800carguru.me` resolves to Elastic IP

Step 1.3: Verify Main Server Protections

- [ ] Confirm **Stop Protection** is enabled
- [ ] Confirm **Termination Protection** is enabled
- [ ] Confirm **IMDSv2** is set to required
- [ ] Note down:
  - Main Instance ID: _____
  - Main Private IP: _____

Step 1.4: Install AWS CLI on Main Server

```
sudo apt install -y awscli
```

Step 1.5: Create AMI of Main Server

- [ ] EC2 Console → Select main instance
- [ ] **Actions** → **Image and templates** → **Create image**

- [ ] Image name: `CG-failover-base-YYYYMMDD`
- [ ] Enable **No reboot**
- [ ] Click **Create image**
- [ ] Wait for AMI status: `available` (10-20 minutes)
- [ ] Note down AMI ID: _____

Phase 1 Checklist:

- [ ] Elastic IP assigned and DNS updated
- [ ] Protections verified
- [ ] AMI created successfully
- [ ] All IDs documented

Phase 2: Launch Backup Server (Day 1 — Afternoon)

> Goal: Create an identical standby server from the AMI.

Step 2.1: Launch Backup Instance from AMI

- [ ] Go to **AMIs** → Select `CG-failover-base-YYYYMMDD`
- [ ] Click **Launch instance from AMI**
- [ ] Configure:

Setting	Value
Name	`SMF_BACKUP_CG`
Instance type	`t3.large` (cost-saving) or `t3.2xlarge` (full mirror)
Key pair	Same as main instance
VPC	`vpc-0423103401dfdca5f`
Subnet	us-east-1f (same AZ)
Auto-assign Public IP	Enable
Security group	Same as main instance
Storage	500 GiB gp3, encrypted

Step 2.2: Enable Protections on Backup

- [ ] **Advanced Details** during launch:
  - Termination protection: **Enable**
  - Stop protection: **Enable**
  - IMDSv2: **Required**

Step 2.3: Launch and Verify

- [ ] Click **Launch instance**
- [ ] Wait for status: **Running**, **2/2 checks passed**

- [ ] SSH into backup: `ssh -i key.pem ubuntu@<backup-public-ip>`
- [ ] Verify Apache is running: `sudo systemctl status apache2`
- [ ] Verify MariaDB is running: `sudo systemctl status mariadb`
- [ ] Note down:
  - Backup Instance ID: _____
  - Backup Private IP: _____
  - Backup Public IP: _____

Step 2.4: Update Security Group for Replication

- [ ] EC2 → **Security Groups** → Select the security group
- [ ] Add **Inbound Rule**:

Type	Port	Source	Description
MySQL/Aurora	3306	Main Private IP/32	DB replication from main
MySQL/Aurora	3306	Backup Private IP/32	DB replication from backup

Phase 2 Checklist:

- [ ] Backup instance launched and running
- [ ] SSH access verified
- [ ] Apache and MariaDB running
- [ ] Security group updated for replication
- [ ] All IDs documented

Phase 3: Database Replication (Day 2 — Morning)

> Goal: Set up real-time database sync from Main (Master) to Backup (Slave).

Step 3.1: Configure Main Server as Master

SSH into **main** server:

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

- [ ] Add under `[mysqld]`:

```
server-id = 1
log_bin = /var/log/mysql/mysql-bin.log
binlog_do_db = carguru2
bind-address = 0.0.0.0
```

- [ ] Restart MariaDB:

```
sudo systemctl restart mariadb
```

Step 3.2: Create Replication User on Main

```
sudo mysql -e "
CREATE USER 'replicator'@'%' IDENTIFIED BY '<STRONG_PASSWORD>';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'%';
FLUSH PRIVILEGES;
"
```

- ☐ Choose a strong password: _____

Step 3.3: Get Master Status

```
sudo mysql -e "SHOW MASTER STATUS;"
```

- ☐ Note down:
  - File: _____
  - Position: _____

Step 3.4: Configure Backup Server as Slave

SSH into **backup** server:

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

- ☐ Add under `[mysqld]`:

```
server-id = 2
relay-log = /var/log/mysql/mysql-relay-bin.log
read_only = 1
```

- ☐ Restart MariaDB:

```
sudo systemctl restart mariadb
```

Step 3.5: Start Replication on Backup

```

```

```
sudo mysql -e "
CHANGE MASTER TO
  MASTER_HOST='<MAIN_PRIVATE_IP>',
  MASTER_USER='replicator',
  MASTER_PASSWORD='<STRONG_PASSWORD>',
  MASTER_LOG_FILE='<FILE_FROM_STEP_3.3>',
  MASTER_LOG_POS=<POSITION_FROM_STEP_3.3>;
START SLAVE;
"
```

Step 3.6: Verify Replication

```
sudo mysql -e "SHOW SLAVE STATUS\G" | grep -E "Slave_IO|Slave_SQL|Seconds_Behind"
```

- [ ] Confirm:
  - `Slave_IO_Running: Yes`
  - `Slave_SQL_Running: Yes`
  - `Seconds_Behind_Master: 0`

Step 3.7: Test Replication

On **main** server:

```
sudo mysql carguru2 -e "CREATE TABLE repl_test (id INT, msg VARCHAR(50)); INSERT
INTO repl_test VALUES (1, 'replication works');"
```

On **backup** server:

```
sudo mysql carguru2 -e "SELECT * FROM repl_test;"
```

- [ ] Verify result shows: `1 | replication works`

Cleanup on **main**:

```
sudo mysql carguru2 -e "DROP TABLE repl_test;"
```

- [ ] Verify table is also removed from backup

Phase 3 Checklist:

- [ ] Main configured as Master
- [ ] Replication user created

- [ ] Backup configured as Slave
- [ ] Replication running (IO + SQL = Yes)
- [ ] Test data replicated successfully
- [ ] Test data cleanup replicated

Phase 4: Auto-Failover Script (Day 2 — Afternoon)

> Goal: Automate detection of main server failure and switch traffic to backup.

Step 4.1: Install and Configure AWS CLI on Backup

```
sudo apt install -y awscli
aws configure
```

- [ ] Enter:
  - AWS Access Key ID: _____
  - AWS Secret Access Key: _____
  - Default region: `us-east-1`
  - Output format: `json`

Step 4.2: Create IAM User for Failover (Least Privilege)

In **AWS Console** → **IAM** → **Users** → **Create user**:

- [ ] Username: `failover-agent`
- [ ] Attach policy — create custom policy:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ec2:AssociateAddress",
                "ec2:DescribeAddresses",
                "ec2:DescribeInstances"
            ],
            "Resource": "*"
        }
    ]
}
```

- [ ] Create **Access Keys** for this user
- [ ] Use these keys in `aws configure` on backup server

Step 4.3: Create Failover Script

```
sudo nano /home/ubuntu/failover.sh
```

- [ ] Paste the script (replace all placeholders):

```
#!/bin/bash

# ============================================
# AUTO-FAILOVER SCRIPT
# Runs every minute via cron on BACKUP server
# ============================================

MAIN_IP="<MAIN_PRIVATE_IP>"
ELASTIC_IP="<YOUR_ELASTIC_IP>"
INSTANCE_ID="<BACKUP_INSTANCE_ID>"
ALLOCATION_ID="<ELASTIC_IP_ALLOCATION_ID>"
FAILOVER_FLAG="/tmp/failover_active"
LOG_FILE="/var/log/failover.log"

# If failover already active, skip
if [ -f "$FAILOVER_FLAG" ]; then
    exit 0
fi

# Check 1: Ping
if ping -c 3 -W 2 $MAIN_IP > /dev/null 2>&1; then
    exit 0
fi

# Check 2: Wait and retry (avoid false positive)
sleep 30

if ping -c 3 -W 2 $MAIN_IP > /dev/null 2>&1; then
    exit 0
fi

# Check 3: HTTP check
if curl -s --max-time 5 http://$MAIN_IP > /dev/null 2>&1; then
    exit 0
fi

# === FAILOVER ===
echo "$(date) - FAILOVER TRIGGERED - Main server DOWN" >> $LOG_FILE

aws ec2 associate-address \
    --instance-id $INSTANCE_ID \
    --allocation-id $ALLOCATION_ID \
    --allow-reassociation \
    --region us-east-1

if [ $? -eq 0 ]; then
```

```
    echo "$(date) - Elastic IP moved to backup" >> $LOG_FILE
    sudo mysql -e "STOP SLAVE; SET GLOBAL read_only = 0;"
    echo "$(date) - Database writable" >> $LOG_FILE
    touch $FAILOVER_FLAG
    echo "$(date) - FAILOVER COMPLETE" >> $LOG_FILE
else
    echo "$(date) - ERROR: Failed to move Elastic IP" >> $LOG_FILE
fi
```

Step 4.4: Set Permissions and Schedule

```
sudo chmod +x /home/ubuntu/failover.sh
sudo crontab -e
```

- ☐ Add:

```
* * * * * /home/ubuntu/failover.sh
```

Step 4.5: Create Failover Log

```
sudo touch /var/log/failover.log
sudo chmod 666 /var/log/failover.log
```

Phase 4 Checklist:

- ☐ AWS CLI configured on backup
- ☐ IAM user created with minimal permissions
- ☐ Failover script created with correct values
- ☐ Cron job scheduled (every minute)
- ☐ Log file created

Phase 5: Daily Backups (Day 2 — Evening)

> Goal: Set up automated daily database backups with local download.

Step 5.1: Create Backup Script on Main Server

```
sudo nano /home/ubuntu/db_backup.sh
```

- ☐ Paste:

```
#!/bin/bash
BACKUP_DIR="/home/ubuntu/db_backups"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
mkdir -p $BACKUP_DIR

mysqldump -u root carguru2 | gzip > "$BACKUP_DIR/carguru2_$DATE.sql.gz"

# Keep only last 7 days
find $BACKUP_DIR -name "*.sql.gz" -mtime +7 -delete

echo "$(date) - Backup completed: carguru2_$DATE.sql.gz" >> /var/log/db_backup.log
```

```
sudo chmod +x /home/ubuntu/db_backup.sh
```

Step 5.2: Schedule Daily Backup (2 AM)

```
sudo crontab -e
```

- ☐ Add:

```
0 2 * * * /home/ubuntu/db_backup.sh
```

Step 5.3: Create Auto-Download Script on Windows PC

Create `D:\DB_BKP\auto_backup.bat`:

```
@echo off
set KEY="C:\path\to\your-key.pem"
set SERVER=ubuntu@<ELASTIC_IP>
set DEST=D:\DB_BKP\

echo Downloading latest backups...
scp -i %KEY% %SERVER%:/home/ubuntu/db_backups/*.sql.gz %DEST%
echo Download complete: %date% %time% >> %DEST%download_log.txt
```

Step 5.4: Schedule on Windows Task Scheduler

- ☐ Open **Task Scheduler**
- ☐ **Create Basic Task**
  - Name: `CG2 Database Backup Download`
  - Trigger: **Daily** at **3:00 AM**
  - Action: **Start a program** → `D:\DB_BKP\auto_backup.bat`

- ☐ Enable: **Run whether user is logged on or not**

Step 5.5: Test Backup Pipeline

```
# On main server — run manual backup
sudo /home/ubuntu/db_backup.sh

# Verify backup created
ls -la /home/ubuntu/db_backups/
```

- ☐ On Windows — run `auto_backup.bat` manually, verify file appears in `D:\DB_BKP\`

Phase 5 Checklist:

- ☐ Backup script created on main server
- ☐ Cron job scheduled (daily 2 AM)
- ☐ Windows download script created
- ☐ Windows Task Scheduler configured
- ☐ Manual test successful

Phase 6: Testing & Validation (Day 3)

> Goal: Verify the entire failover system works end-to-end.

Step 6.1: Pre-Test Checks

- ☐ Main server is running and serving traffic
- ☐ Backup server is running
- ☐ Replication status: `Slave_IO_Running: Yes`, `Slave_SQL_Running: Yes`
- ☐ Failover cron is active on backup
- ☐ Inform team about planned test downtime (5 minutes)

Step 6.2: Test Failover (Simulate Main Failure)

Option A — Stop Main Instance:

1. ☐ Disable stop protection on main temporarily
2. ☐ Stop the main instance from EC2 console
3. ☐ Start timer

Expected Result:

- ☐ Within 1-2 minutes, failover script activates
- ☐ Elastic IP moves to backup
- ☐ Website `800carguru.me` loads from backup server
- ☐ Database is writable on backup
- ☐ Check `/var/log/failover.log` on backup for entries

Step 6.3: Verify Failover Success

```
# On backup server
cat /var/log/failover.log
ls /tmp/failover_active
sudo mysql -e "SHOW VARIABLES LIKE 'read_only';"
```

- [ ] Log shows failover triggered
- [ ] Flag file exists
- [ ] `read_only: OFF`

Step 6.4: Test Application During Failover

- [ ] Login to `800carguru.me`
- [ ] Dashboard loads correctly
- [ ] Can create/edit a test record
- [ ] WhatsApp section accessible

Step 6.5: Test Failback

1. [ ] Start the main instance
2. [ ] Wait for it to be fully running
3. [ ] Perform failback (see Phase 7 below)
4. [ ] Verify website serves from main again
5. [ ] Verify replication resumes on backup

Step 6.6: Record Test Results

Test	Result	Notes
Failover triggered	Pass / Fail	
Elastic IP moved	Pass / Fail	
Website accessible	Pass / Fail	
Database writable	Pass / Fail	
Application functional	Pass / Fail	
Failback successful	Pass / Fail	
Replication resumed	Pass / Fail	
Total downtime	___ minutes	

Phase 6 Checklist:

- [ ] Failover test completed successfully
- [ ] Failback test completed successfully
- [ ] All test results documented

- [ ] Re-enable stop protection on main

Phase 7: Failback Procedure Reference

> This is not a setup phase — keep this as a reference for when failback is needed.

Step 7.1: Ensure Main Server is Healthy

```
# SSH into main
sudo systemctl status apache2
sudo systemctl status mariadb
```

Step 7.2: Sync Data from Backup to Main

On **backup** server:

```
sudo mysql -e "SHOW MASTER STATUS;"
```

Note: File = ___, Position = ___

On **main** server:

```
sudo mysql -e "
CHANGE MASTER TO
  MASTER_HOST='<BACKUP_PRIVATE_IP>',
  MASTER_USER='replicator',
  MASTER_PASSWORD='<STRONG_PASSWORD>',
  MASTER_LOG_FILE='<FILE>',
  MASTER_LOG_POS=<POSITION>;
START SLAVE;
"
```

Wait for sync:

```
sudo mysql -e "SHOW SLAVE STATUS\G" | grep "Seconds_Behind_Master"
```

Wait until: `Seconds_Behind_Master: 0`

Step 7.3: Switch Elastic IP Back to Main

```
aws ec2 associate-address \
    --instance-id <MAIN_INSTANCE_ID> \
```

```
    --allocation-id <ELASTIC_IP_ALLOCATION_ID> \
    --allow-reassociation \
    --region us-east-1
```

Step 7.4: Restore Replication Direction

On **main**:

```
sudo mysql -e "STOP SLAVE; RESET SLAVE ALL;"
sudo mysql -e "SHOW MASTER STATUS;"
```

Note: File = ___, Position = ___

On **backup**:

```
sudo mysql -e "
SET GLOBAL read_only = 1;
CHANGE MASTER TO
  MASTER_HOST='<MAIN_PRIVATE_IP>',
  MASTER_USER='replicator',
  MASTER_PASSWORD='<STRONG_PASSWORD>',
  MASTER_LOG_FILE='<FILE>',
  MASTER_LOG_POS=<POSITION>;
START SLAVE;
"
```

Step 7.5: Remove Failover Flag

On **backup**:

```
rm /tmp/failover_active
```

Step 7.6: Verify Everything

- ☐ Website serves from main
- ☐ Replication running on backup (`Slave_IO: Yes`, `Slave_SQL: Yes`)
- ☐ Failover cron still active on backup
- ☐ `/tmp/failover_active` removed

Post-Setup: Monthly Maintenance

Task	Frequency	Command / Action
Check replication status	Weekly	`SHOW SLAVE STATUS\G` on backup

Task	Frequency	Command / Action
Failover drill	Monthly	Stop main, verify auto-switch
Review failover logs	Weekly	`cat /var/log/failover.log`
Verify backups exist	Weekly	`ls /home/ubuntu/db_backups/`
Verify local downloads	Weekly	Check `D:\DB_BKP\`
Update AMI snapshot	Monthly	Create new AMI from main
Review security groups	Monthly	Ensure no unnecessary open ports
Check disk space	Weekly	`df -h` on both servers

Quick Reference Card

Item	Value
Main Instance ID	_____
Backup Instance ID	_____
Elastic IP	_____
Elastic IP Allocation ID	_____
Main Private IP	_____
Backup Private IP	_____
Replication User	`replicator`
Replication Password	_____
Domain	`800carguru.me`
Database	`carguru2`
VPC	`vpc-0423103401dfdca5f`
Region / AZ	`us-east-1f`
Key Pair	_____
Failover Log	`/var/log/failover.log`
Backup Directory	`/home/ubuntu/db_backups/`
Local Backup	`D:\DB_BKP\`

Emergency Contacts

Role	Name	Phone	Email
IT Lead			

Role	Name	Phone	Email
AWS Account Owner			
Domain Manager			

Document Version: 1.0 **Last Updated:** April 1, 2026